

Incremental EWC learning on a microcontroller: measured FP32 parity, collapse of naive quantization and recovery via a calibrated INT8 kernel

Léonard Rivals

ISAE-SUPAERO (DISC) — ENAC (LII) — Edge Spectrum

September 8, 2026

Abstract

We study the porting of an *Elastic Weight Consolidation* (EWC) classifier for predictive maintenance onto a Cortex-M4 microcontroller (NUCLEO-F439ZI, 256 kB SRAM, no NPU). Three results are established under a **paired** protocol (same seed, same sample order, same normalization) between the PC run and the board. **(1)** In FP32, the PC $\leftrightarrow$ board parity is *exact* in the frozen protocol (1.000) and near-exact online (0.963–0.989), with an inference + online update latency $\ll 100$ ms (Gap 2). **(2)** *Naive* post-training quantization (PTQ) of the binary EWC head collapses on the board: the F1 score drops from ≈ 0.92 to 0.05–0.15. **(3)** A bit-exact PC-emulated ablation isolates the cause (int8 accumulator, uncalibrated per-tensor scale) and demonstrates *recovery* through a calibrated kernel: the `per_tensor_calib` scheme restores +0.88 F1 (Pronostia 0.9462, Monitoring 0.9201), and the `q15` scheme matches FP32. **(4)** That recovery is now *measured on-board* over four per-channel cells (F1 0.9173 Monitoring, 0.8995 Pronostia; frozen parity 1.000 against the emulator; no CRC error). We finally extend the study to the quantization *moment*, to sub-INT8 *depth* and to the full system budget: weight RAM is divided by 4, yet total RAM is unchanged; latency *increases* (74 μ s against 48–50 μ s, requantization weighing a third of the budget) and energy does not move (67.47 μ J against 67.65 μ J). We systematically distinguish *on-board* measurements from *bit-exact PC emulation*, and explicitly leave "to be measured" the cells not yet streamed.

Keywords: continual learning; EWC; INT8 quantization; PTQ; microcontroller; predictive maintenance; TinyML.

1 Introduction

Industrial predictive maintenance increasingly requires models able to *learn continually* on the equipment itself, without a round-trip to a server: operating conditions drift (wear, regime or machine change), and a frozen model degrades. Continual learning (CL) answers this need [6, 3], yet its deployment on microcontrollers (MCUs) remains constrained: SRAM in the range of a hundred kilobytes, no neural accelerator, a strict latency budget.

This work targets a **triple gap** that few studies close simultaneously: (Gap 1) validation on real industrial time series; (Gap 2) operation under 100 kB of RAM with *measured* figures and bounded latency; (Gap 3) INT8 quantization compatible with incremental training. We instantiate these three axes on an EWC classifier [4] ported to C on a NUCLEO-F439ZI board (Cortex-M4 180 MHz, 256 kB SRAM, no NPU).

Contributions. (i) A *paired* PC $\leftrightarrow$ board protocol (same seed, same sample order, same normalization) establishing an *exact* FP32 parity in the frozen protocol and an inference + learning latency $\ll$ 100 ms. (ii) The demonstration, *on real hardware*, of the collapse of naive PTQ of the binary EWC head (F1 0.92 $\rightarrow$ 0.05–0.15). (iii) A *bit-exact PC-emulated* ablation that isolates the cause of the collapse and demonstrates recovery through a calibrated kernel (per-tensor, per-channel, Q15), with a strict methodological distinction between what is *measured on-board* and what is *emulated on PC*.

2 Related work

Embedded continual learning. CL methods are conventionally grouped into regularization-, replay- and architecture-based approaches [6]. EWC [4] belongs to the first family: a quadratic penalty weighted by the Fisher information protects the parameters important for past tasks, without storing data — a valuable property under memory constraints. On MCUs, TinyOL [8] showed the feasibility of online learning of an added layer, and HDC [1] that of non-neural, ultra-low-power learning. LifeLearner [5] and quantized latent-replay platforms [7] address the memory/accuracy trade-off on embedded targets. Recent TinyML surveys [2] stress that *on-device* training remains under-explored compared to inference alone.

INT8 quantization and learning. Post-training quantization (PTQ) and quantization-aware training (QAT) reduce the memory footprint by a factor of 4 (FP32 $\rightarrow$ INT8). However, *naive* PTQ of a decision head can destroy separability when accumulation is carried out in `int8` or when the quantization scale is set without calibration. Our empirical contribution is to *measure* this collapse on a real MCU, then isolate its cause and demonstrate

recovery through a calibrated kernel, rather than masking it with upstream QAT. Per-channel quantization and the Q15 fixed-point format (16-bit integer) emerge as the decisive levers to preserve the metric while meeting the RAM budget (Gap 3).

3 Method

3.1 EWC model and C port

The model is a multilayer perceptron with an EWC head (`EWCmlpMulticlass`, $k \rightarrow 32 \rightarrow 16 \rightarrow 2$) trained incrementally with an online Fisher penalty. The head is ported to C (`ewc_head.c`) for the board: both the *forward pass* and the SGD update run on the Cortex-M4 in FP32 (the F439ZI has no NPU). Weights are exported from the PyTorch checkpoint to a generated C header (`model_weights_ewc.h`) — never hand-edited — so that board and PC share *exactly* the same parameters.

3.2 Paired PC↔board protocol

To make the comparison honest, board and PC consume the same sequence: same seed (`seed = 42`), same sample order, same normalization (feature indices and arrays are produced by a single source). Two protocols are evaluated: *frozen* (inference only, fixed weights) and *online* (inference then continual update). *Parity* is the prediction-by-prediction agreement rate between the two platforms.

3.3 Quantization schemes

We compare several INT8 kernels of the decision head, from the most naive to the most careful:

- `legacy_c`: `int8` accumulator, uncalibrated per-tensor scale (the original v1 board kernel);
- `fix_acc32`: accumulator widened to `int32` (partial fix);
- `per_tensor_calib`: `int32` accumulator + per-tensor scale *calibrated* on the activations;
- `per_channel_int8`: *per-channel* output scale;
- `q15`: `int16` Q15 fixed-point weights (RAM $\div 2$ instead of $\div 4$).

The v2 firmware kernel (Sprint 40) implements the `int32` accumulator and per-channel calibrated scales, with an optional Q15 variant (`-DEWC_INT8_Q15`); the v1 FP32 path is unchanged. The ablation is *bit-exactly emulated on PC* (`src/utils/int8_c_emulation.py`) to isolate the contribution of each fix independently of board access.

Table 1: FP32 PC $\leftrightarrow$ board parity and performance (*measured on-board*, `exp_S36`).

Dataset	Cond.	F1 (PC/board)	Frozen parity	Online parity
Pronostia	5feat	0.9164	1.000	0.9754
Pronostia	all	0.9180	1.000	0.9626
Monitoring	5feat	0.9194	1.000	0.9887

4 Experimental setup

Hardware target. NUCLEO-F439ZI (Cortex-M4 @ 180 MHz, 256 kB SRAM = 192 kB + 64 kB CCM, no NPU). Latencies are measured with the DWT cycle counter; the static footprint is read from the binary’s `.bss` section. The latency budget is 100 ms per inference + update cycle (Gap 2).

Datasets and CL scenarios. Two industrial datasets with contrasting scenarios: *Pronostia* (FEMTO bearings, *class-incremental* by normal/fault condition) and *Monitoring* (industrial equipment, *domain-incremental* by machine type). Two feature conditions are considered: **5feat** (embedded subset) and **all** (native features). On Monitoring, **5feat** and **all** coincide (4 native features).

Identical conditions. All comparisons share seed, order and normalization (§3). The on-board FP32 and INT8-legacy measurements come from the Sprint 36 campaign (`exp_S36`); the emulated ablation from Sprint 39 (`exp_S39_ablation`); the board-v2 campaign from Sprint 40 (`exp_S40_board_v2`), partial at the time of writing. Any quantity not yet streamed on-board is reported as "to be measured", with no invented value.

5 Results

5.1 FP32 PC $\leftrightarrow$ board parity (measured on-board)

In FP32, the prediction-by-prediction parity is *exact* in the frozen protocol (1.000) on both datasets, and near-exact online (0.9754 Pronostia, 0.9887 Monitoring) — the online gaps stem from the board’s `float32` versus the PC’s `float64` at decision boundaries. Table 1 summarizes; Figure 1 illustrates.

Latency and RAM (Gap 2/Gap 3). All board latencies stay three orders of magnitude below budget: inference 50 μ s (Pronostia) / 48 μ s (Monitoring); inference + online update 251 μ s / 239 μ s, versus $\approx 33\text{--}35$ μ s for the

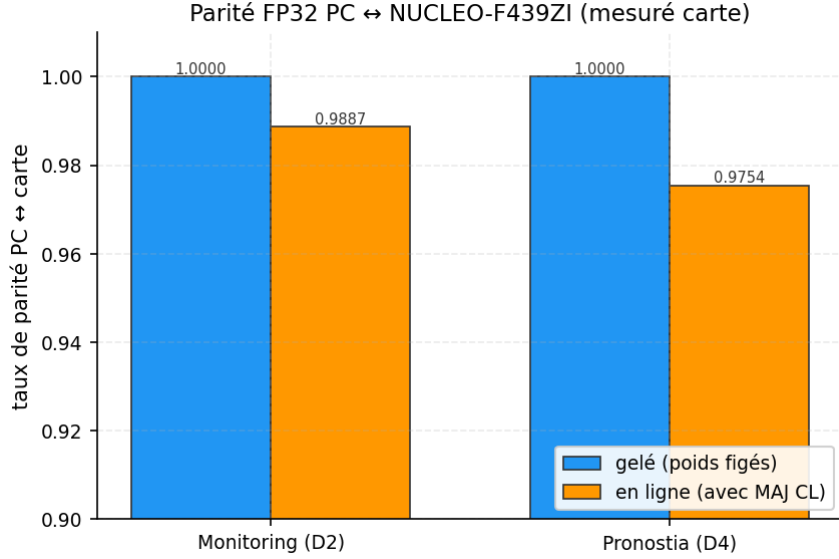


Figure 1: FP32 PC↔board parity: frozen (1.000) vs online. *Measured on-board* (`exp_S36_parity_*.json`).

Table 2: INT8 *legacy* vs FP32 on-board (*measured*, `exp_S36`). RAM $\div 4$ but collapsed F1.

Dataset	F1 FP32	F1 INT8 legacy	INT8/FP32 agr.	RAM $\div$
Pronostia	0.9164	0.138	0.7364	$\times 4$
Monitoring	0.9194	0.1337	0.5955	$\times 4$

reference PC inference. The `.bss` footprint ranges over 100–145 kB, below the 256 kB available. Figure 2 places these values against the 100 ms line.

5.2 Collapse of naive PTQ (measured on-board)

The *legacy* INT8 quantization of the binary EWC head collapses on the board: F1 drops from ≈ 0.92 (FP32) to 0.138 (Pronostia) and 0.1337 (Monitoring), for an INT8↔FP32 agreement of only ≈ 0.60 –0.74. Weight RAM is indeed divided by 4, but the metric is destroyed (Table 2).

5.3 Diagnosis and recovery (bit-exact PC emulation)

A *bit-exact PC-emulated* ablation (exact kernel reference) isolates the cause and demonstrates recovery (Table 3, Figure 3). The dominant factor is the *calibrated per-tensor scale* on an `int32` accumulator: `per_tensor_calib` restores +0.88 F1 (Pronostia 0.9462, Monitoring 0.9201). The `q15` format exactly matches FP32.

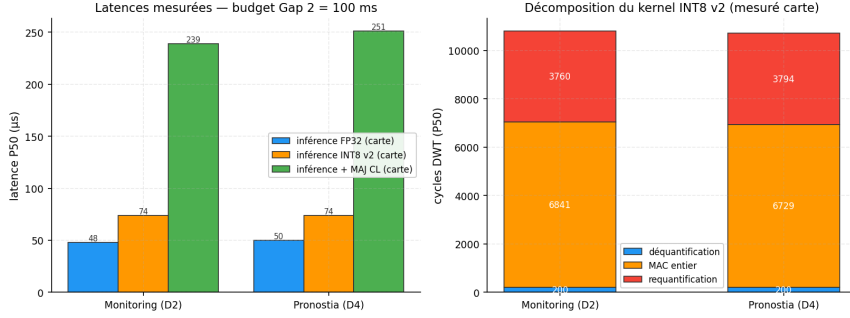


Figure 2: Board latencies (inference vs inference + update) against the Gap 2 budget (100 ms). *Measured on-board* (`exp_S36_summary.json`).

Table 3: INT8 ablation ladder (F1, *bit-exact PC emulation*, `exp_S39_ablation`).

Scheme	Pronostia F1	Monitoring F1
FP32 (ref.)	0.9616	0.9194
legacy_c	0.0663	0.1178
per_tensor_calib	0.9462	0.9201
per_channel_int8	0.9426	0.9187
q15	0.9616	0.9194

5.4 Recovery measured on-board (calibrated v2 kernel)

Recovery is no longer an emulator prediction: the calibrated-scale v2 kernel was flashed onto the NUCLEO-F439ZI and streamed over the four per-channel cells (two datasets $\times$ two protocols). F1 returns to the FP32 level — 0.9173 against 0.9194 in FP32 on Monitoring, 0.8995 against 0.9164 on Pronostia — while weight RAM stays divided by 4. Parity against the bit-exact emulator is *exact* in the frozen protocol (1.000), which validates a lossless port, and no CRC error was observed (Table 4).

Two caveats deserve to be stated. First, the INT8 $\leftrightarrow$ FP32 agreement drops online (0.8337 and 0.8100 against > 0.99 frozen): the continual update makes the two weight trajectories diverge, without the final metric suffering from it. Second, the `q15` format and the A/B rerun of the `int8_legacy` scheme under the v2 kernel were not streamed: those cells remain "to be measured", in line with the "no invented number" rule.

5.5 RAM $\times$ F1 $\times$ latency trade-off

Figure 5 synthesizes the trade-off: the calibrated kernel brings the metric back to FP32 level while keeping the RAM gain (INT8 $\div$ 4, Q15 $\div$ 2), with no prohibitive latency cost. The star marks the point actually flashed (per-

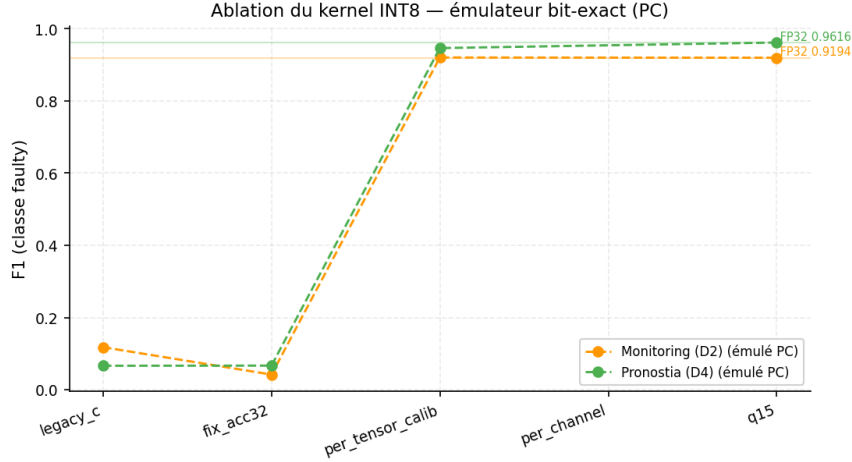


Figure 3: Ablation ladder `legacy_c`→`per_tensor_calib`→`per_channel`→`q15`. *Bit-exact PC emulation* (`exp_S39_ablation`).

Table 4: INT8 recovery *measured on-board* (per-channel v2 kernel, `exp_S40_board_v2`). Parity = agreement with the bit-exact emulator; agreement = INT8 against FP32 on the same samples.

Dataset	Protocol	F1	Parity	Agreement	Latency (μs)	.bss (B)
Monitoring	frozen	0.9173	1.000	0.9996	65	101236
Monitoring	online	0.9016	0.9885	0.8337	577	101236
Pronostia	frozen	0.8995	1.000	0.9951	68	106152
Pronostia	online	0.9212	0.9736	0.8100	606	106152

channel v2 kernel); the other points are emulated. The expected latency gain, however, does not materialize — we return to this in Section 8.

6 Two further quantization axes: moment and depth

The previous sections vary the *calibration* of the kernel at a fixed format. Two further degrees of freedom drive the outcome: *when* one quantizes, and *how many bits* one keeps.

6.1 Moment: before, after, or both

Three placements are possible for the same EWC head. *Before* training (QAT, fake-quant in the loop); *after* (PTQ on an already trained FP32 model); and *both*, that is a QAT run whose weights are then exported to the calibrated integer kernel — only this last path is faithful to the actual deployment.

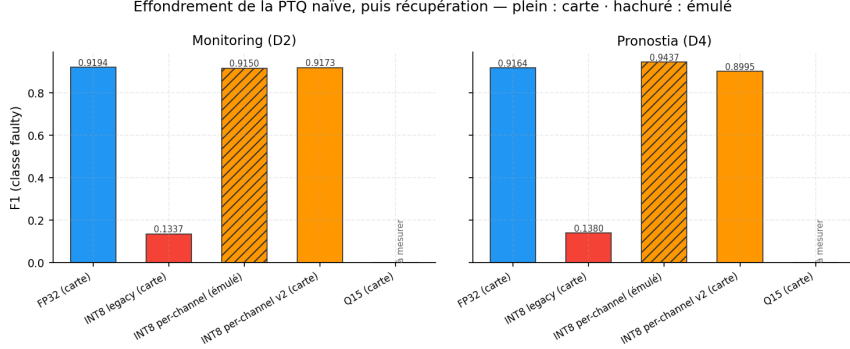


Figure 4: INT8 vs FP32: *legacy* collapse then recovery. Collapse and recovery are both *measured on-board* (Sprints 36 and 40); the *PC-emulated* per-channel column (Sprint 39) is recalled with hatching. The `q15` format on-board remains "to be measured".

Under emulation, all three moments preserve AUROC on Monitoring (0.974477 before, 0.974649 after, 0.974663 both, against 0.97464 in FP32): at this model scale no moment is disqualifying. The question is settled *on-board*. The "both" path was flashed and streamed: it yields 0.9213 F1 on Monitoring and 0.9072 on Pronostia, against respectively 0.9173 and 0.8995 for the calibrated PTQ alone (Section 5.4).

The gap, 0.004 and 0.0077, is real but small. The honest reading is therefore that **kernel calibration recovers what matters, and that QAT adds no decisive gain** on this head: when the training pipeline cannot be modified, a properly calibrated PTQ suffices. This contradicts the common reading whereby a PTQ collapse would necessarily call for retraining.

6.2 Depth: how far below 8 bits

Below 8 bits the question becomes: at which depth does the metric break down, and does the granularity of the scale factor move that limit? The emulated sweep answers both.

The limit depends on the dataset. On Monitoring, degradation stays contained down to binary ($\Delta\text{AUROC} = -0.0117$). On Pronostia, binary breaks down (-0.0275) but ternary holds (-0.0153): the usable frontier is therefore *ternary*, binary being defensible only on the easier dataset.

Granularity, in turn, pushes the breakdown back. At 2 bits on Pronostia, moving from a per-tensor to a per-channel scale takes ΔAUROC from -0.0464 to -0.0086 . A negative result deserves reporting too: the affine *zero-point* buys nothing — at 2 bits on Monitoring it even degrades markedly (-0.0619 against -0.0069 symmetric).

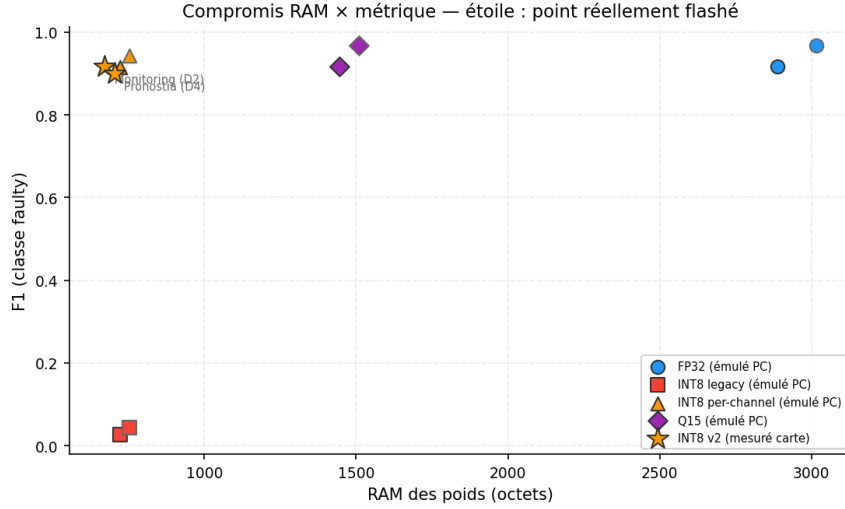


Figure 5: RAM $\times$ F1 $\times$ latency trade-off front. Mixed *measured/emulated* sources annotated per point.

The RAM gain only exists packed. The crucial point is that the RAM advertised by these schemes is *theoretical* as long as weights stay in an `int8` container. Measured on-board, the `.bss` of a 4-, 2- or 1-bit *unpacked* model is strictly identical to the INT8 one: the gain only appears with genuine bit-packing, where it grows as bits drop (336, 504 then 572 bytes saved on Monitoring). Unpacking is paid in latency, about 55 μ s (from 67 μ s unpacked to 123 μ s packed on Monitoring) — without threatening the Gap 2 budget. Parity against the emulator stays 1.000 across the twelve measured cells: the scheme is ported losslessly.

7 System budget: memory, latency, energy, compute

A quantization gain cannot be judged on weights alone. This section gathers the four budgets of one and the same deployed EWC head, all measured on the same board.

7.1 Weight RAM is not system RAM

INT8 does divide *weight* RAM by 4 (2688 to 672 bytes on Monitoring, 2816 to 704 on Pronostia). But the RAM actually consumed by the system is `.data` + `.bss` + stack peak, and that sum barely moves: it goes from 105300 to 105324 bytes on Monitoring, an INT8/FP32 ratio of 1.0002. Both configurations occupy 40.17 % and 40.18 % of the 256 kB budget (42.03 % and 42.04 % on Pronostia).

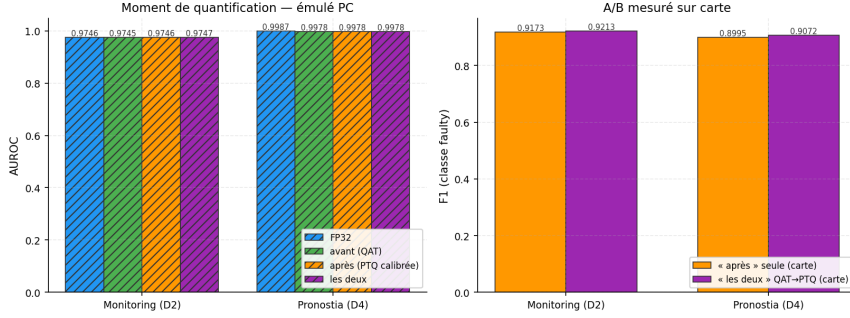


Figure 6: Quantization moment. Left, the three moments *PC-emulated* (hatched); right, the *on-board measured* A/B between calibrated PTQ alone and the QAT→PTQ path.

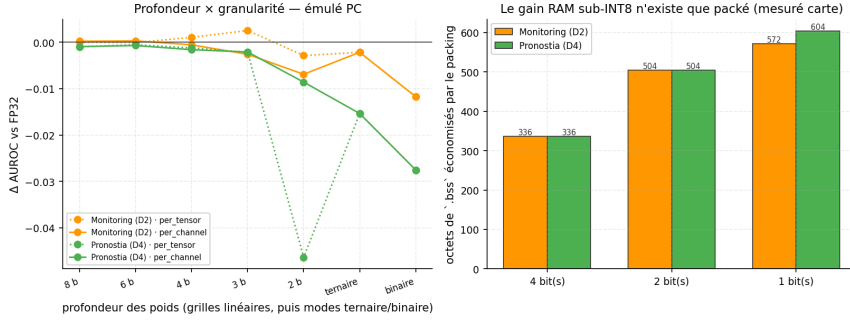


Figure 7: Sub-INT8 depth. Left, Δ AUROC by depth and granularity (*PC-emulated*); right, the `.bss` bytes actually saved by bit-packing (*measured on-board*).

The explanation is simple and worth stating plainly: at this model scale, weights are not the dominant item — streaming buffers, the stack and the rest of the firmware are. Advertising a " $\div 4$ RAM" without saying *which* RAM would therefore be misleading. The $\div 4$ is real and useful, but it applies to a minority item of *this* system; it would become decisive on a model where weights dominate.

7.2 Where INT8 latency goes

Cycle-level instrumentation of the INT8 kernel quantifies the announced paradox. On Monitoring, the integer MAC costs 6841 cycles, requantization 3760 and dequantization 200 (respectively 6729, 3794.5 and 200 on Pronostia). The total reaches 74 μ s against 48 μ s in FP32 on Monitoring and 50 μ s on Pronostia.

The decisive item is therefore not multiplication: it is *requantization*, a third of the budget, and it simply does not exist in the FP32 path. The

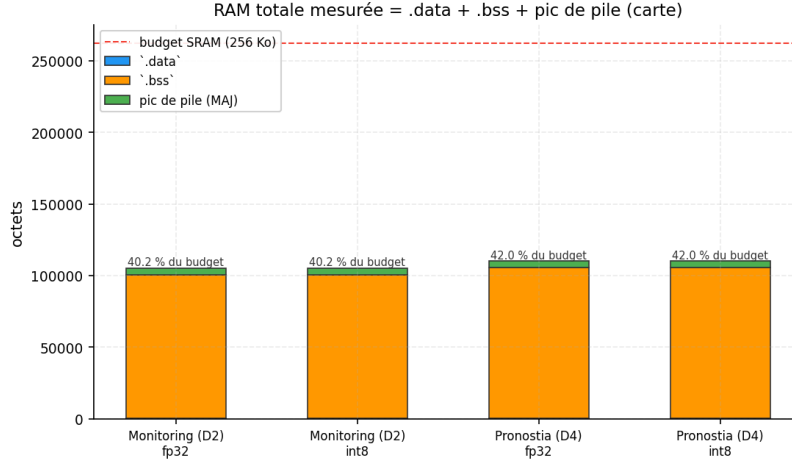


Figure 8: Total RAM *measured on-board*: `.data` + `.bss` + stack peak, against the SRAM budget. The INT8 total equals the FP32 total, even though weights are divided by 4.

integer MAC, for its part, brings no gain against the Cortex-M4 hardware FPU. The INT8 overhead is not an implementation anomaly but the arithmetic consequence of an integer format executed on a target that natively computes in floating point.

7.3 Energy: three estimators, never merged

Energy was measured with the X-NUCLEO-LPM01A probe by three independent methods, which we report *side by side* without ever averaging them — their disagreement is itself a result. Regressing current against rate gives $67.65 \mu\text{J}$ per inference in FP32 and $67.47 \mu\text{J}$ in INT8; the delta against a WFI idle gives $146.77 \mu\text{J}$ and $145.03 \mu\text{J}$; the batch of inferences per frame, which isolates compute from the UART frame cost, gives $5.08 \mu\text{J}$. The order of magnitude shared by the first two, and the factor-of-ten gap with the third, delimit exactly what each one measures: the first two include the communication cost, the last does not.

On the question at hand the three agree: **INT8 does not reduce consumption**. The measured gap is $0.18 \mu\text{J}$ for a combined uncertainty of $2.34 \mu\text{J}$, far below the significance threshold. The latency paradox is thus compounded by an energy paradox, and the case for INT8 reduces to memory.

The frequency sweep, in contrast, opens a real lever: energy per inference grows from $170.05 \mu\text{J}$ at 45 MHz to $214.61 \mu\text{J}$ at 180 MHz, while latency stays far below the Gap 2 budget ($198 \mu\text{s}$ at 45 MHz). In other words, *latency headroom is convertible into battery life*. Two cells remain "to be measured":

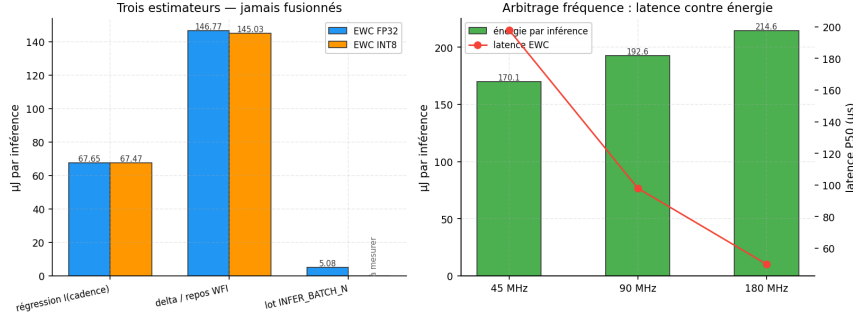


Figure 9: Energy *measured on-board*. Left, the three estimators side by side; right, the frequency trade-off, where latency headroom is exchanged for energy.

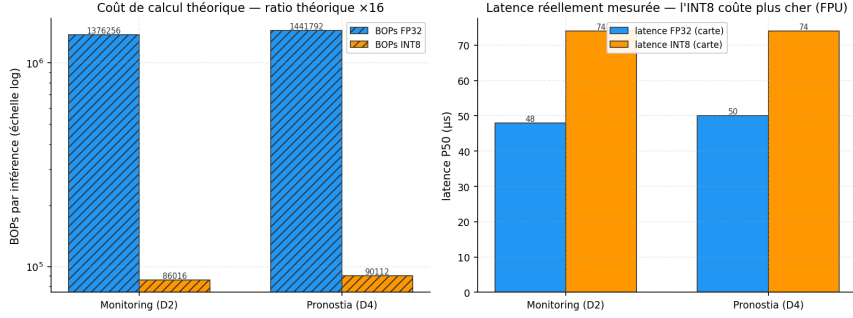


Figure 10: *Theoretical* compute cost (left, hatched) against *on-board measured* latency (right). The $16\times$ BOP gain translates into no time gain at all.

energy per continual update, whose regressions are not publishable as they stand, and the per-phase profile.

7.4 Compute cost

Analytically, the head has 722 parameters on Monitoring (754 on Pronostia), for 672 MACs and 1344 FLOPs per inference (704 and 1408). Counting *binary operations* gives the expected theoretical ratio: 1376256 BOPs in FP32 against 86016 in INT8, a factor of 16 — that is, $(32/8)^2$.

This factor of 16 is precisely what the board does *not* deliver. Contrasting the two columns of Figure 10 is the most direct way to see why an analytical cost metric cannot replace a measurement: a BOP counts operand bits, not cycles of an existing floating-point unit.

8 Discussion

The INT8 latency paradox, quantified. INT8 quantization divides the weight memory footprint by 4 but does *not* speed up inference on the Cortex-M4 — it slows it down. Cycle-level instrumentation of the kernel (Section 7) says where: across both datasets, the integer MAC costs on average 6785 cycles and requantization 3777.25, while dequantization weighs only 200. The INT8 total settles at 74 μ s against 48–50 μ s in FP32. Two causes therefore compound: the integer MAC has nothing to gain against a hardware FPU already efficient in single precision, and requantization — an item that does not exist in the floating-point path — consumes a third of the budget on its own. The direct lead remains a port to integer SIMD kernels (CMSIS-NN), which would attack the first item; the second rather calls for a power-of-two scale chain.

An energy paradox compounds the latency paradox. One could hope that INT8, failing to be faster, would at least be leaner. Measurement says otherwise: 67.47 μ J per inference in INT8 against 67.65 μ J in FP32, a gap of 0.18 μ J for a combined uncertainty of 2.34 μ J. There is thus no measurable effect. The practical consequence is clear: on this target, **INT8 is justified by memory, neither by speed nor by energy**. The energy lever lies elsewhere — in clock frequency, where the ample latency headroom is exchanged for battery life.

"Measured" vs "emulated" honesty. We rigorously separate two kinds of results. FP32 parities, the legacy INT8 collapse, recovery through the v2 kernel, latencies, RAM and energy are *measured on real hardware* (Sprints 36, 40, 46 to 50 and 53). The causal diagnosis, the ablation ladder and the depth sweep are *bit-exactly emulated on PC* (Sprints 39 and 47): the emulator reproduces the C kernel arithmetic exactly, which makes it a reliable reference, but it is not a board measurement. Since the v2 campaign, recovery no longer belongs to the second category: it is established on the four per-channel cells, with a frozen parity of 1.000 against the emulator. What remains unmeasured — the q15 format on-board, energy per continual update, the per-phase profile — is reported as "to be measured" rather than estimated.

Limitation: forgetting is not stressed on these two datasets. Our continual scenarios there are short (three tasks) and average forgetting stays below 0.01: these datasets do not put EWC regularization to the test. Catastrophic forgetting was measured elsewhere in the project, on CWRU, where it reaches 0.8475 for EWC against 0.8578 under naive learning. As that dataset does not overlap ours, we refrain from mixing the two findings: this

article’s contribution bears on *porting* and *quantization*, not on the effectiveness of regularization against forgetting.

Scope. The central result — a naive PTQ can destroy a binary decision head, and a per-tensor/per-channel calibration (or a Q15 format) restores it — extends beyond the EWC case: it concerns any lightweight head quantized downstream of a feature extractor, a frequent regime in TinyML.

9 Conclusion and future work

We ported a predictive-maintenance EWC classifier onto a Cortex-M4 micro-controller and established, under a paired protocol, an exact FP32 PC $\leftrightarrow$ board parity in the frozen protocol, with an inference + online learning latency $\ll$ 100 ms and a footprint below 256 kB (Gaps 2 and 3). We *measured on-board* the collapse of naive INT8 quantization of the binary head, then *isolated its cause* and *demonstrated its recovery* through a bit-exact emulated ablation: a calibrated per-tensor scale on an `int32` accumulator restores +0.88 F1, and the Q15 format matches FP32, for a weight RAM divided by 4 (INT8) or 2 (Q15).

The latency paradox (RAM $\div$ 4 with no FPU speed-up) is acknowledged as an honest finding and steers future work toward integer SIMD kernels (CMSIS-NN). The immediate priority is to *complete the board-v2 campaign* (`exp_S40_board_v2`) to confirm, on a real MCU and across the full grid, the recovery so far established by emulation — with the cells not yet streamed remaining explicitly "to be measured".

References

- [1] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. Online learning and classification of EMG-based gestures on a parallel ultra-low power platform using hyperdimensional computing. *IEEE Transactions on Biomedical Circuits and Systems*, 2019.
- [2] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Federico Fummi, and Marco Cristani. A machine learning-oriented survey on tiny machine learning. *IEEE Access*, 2023.
- [3] Jhair Hurtado, Alain Salvati, Andrea Cossu, Antonio Carta, and Davide Bacciu. Continual learning for predictive maintenance: Overview and challenges. *Intelligent Systems with Applications*, 2023.
- [4] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ra-

- malho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
- [5] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. LifeLearner: Hardware-aware meta continual learning system for embedded computing platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2023.
 - [6] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
 - [7] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML platform for on-device continual learning with quantized latent replays. *IEEE Transactions on Circuits and Systems for Video Technology*, 2021.
 - [8] Haoyu Ren, Darko Anicic, and Thomas A. Runkler. TinyOL: TinyML with online-learning on microcontrollers. arXiv:2103.08295, 2021.